

บทที่ 3

การแก้ปัญหาด้วยวิธีการค้นหาแบบปฏิกิริยา

3.1 บทนำ

ในบทที่ผ่านมาได้กล่าวถึงการแก้ปัญหาด้วยวิธีการค้นหาในสภาพแวดล้อมที่มีลักษณะคงที่ (Static Environment) ซึ่งผลลัพธ์ของการกระทำขึ้นอยู่กับตัวแทน (Agent) เพียงฝ่ายเดียว ตัวอย่างเช่น การค้นหาเส้นทางที่สั้นที่สุด การแก้ปัญหา 8 ตัวเลข หรือการค้นหาเส้นทางในเขาวงกต ในปัญหาเหล่านี้ สภาพแวดล้อมจะไม่เปลี่ยนแปลงไปตามการตัดสินใจของฝ่ายอื่น ทำให้ตัวแทนสามารถวางแผนล่วงหน้าและค้นหาเส้นทางที่เหมาะสมที่สุดได้โดยอาศัยข้อมูลที่มีอยู่

อย่างไรก็ตาม ในปัญหาหลายประเภท การตัดสินใจของตัวแทนไม่ได้ขึ้นอยู่กับตนเองเพียงฝ่ายเดียว แต่ยังขึ้นอยู่กับกรกระทำของฝ่ายตรงข้ามหรือคู่แข่ง ซึ่งพยายามเลือกทางเลือกที่เป็นประโยชน์ต่อตนเอง และส่งผลเสียต่อฝ่ายตรงข้าม สถานการณ์เช่นนี้พบได้บ่อยในเกมกระดาน เกมวางแผน การแข่งขันทางธุรกิจ รวมทั้งการควบคุมหุ่นยนต์ที่ต้องเผชิญกับคู่แข่ง

การแก้ปัญหาในลักษณะดังกล่าวเรียกว่า การค้นหาแบบปฏิกิริยา (Adversarial Search) หรือที่เรียกว่า การค้นหาในเกม (Game Search) ซึ่งเป็นกระบวนการที่ตัวแทนต้องพิจารณาทั้งการกระทำของตนเอง และการตอบสนองที่เป็นไปได้ของคู่ต่อสู้ เพื่อเลือกแนวทางที่ให้ผลลัพธ์ที่ดีที่สุดภายใต้สมมติฐานว่าคู่ต่อสู้จะเลือกการกระทำที่ดีที่สุดสำหรับตนเองเช่นเดียวกัน

ตัวอย่างของปัญหาที่สามารถประยุกต์ใช้การค้นหาแบบปฏิกิริยา ได้แก่

- เกมหมากกรุก (Chess)
- เกมหมากฮอส (Checkers)
- เกมโกะ (Go)
- เกมโอเทลโล (Othello)
- เกมทิก-แทค-โท (Tic-Tac-Toe)
- เกมวางแผน (Strategy Games)

ลักษณะสำคัญของเกมเหล่านี้ คือ ผู้เล่นแต่ละฝ่ายมีเป้าหมายที่ขัดแย้งกัน กล่าวคือ ความสำเร็จของผู้เล่นฝ่ายหนึ่งมักหมายถึงความล้มเหลวของอีกฝ่ายหนึ่ง ดังนั้นการตัดสินใจจึงไม่สามารถพิจารณาจากสถานะปัจจุบันเพียงอย่างเดียว แต่ต้องคำนึงถึงลำดับการเดินที่อาจเกิดขึ้นในอนาคตด้วย

ในบทนี้จะกล่าวถึงหลักการพื้นฐานของการค้นหาแบบปฏิกิริยา การแทนปัญหาในรูปของต้นไม้เกม ขั้นตอนวิธีมินิแมกซ์ (Minimax Algorithm) เทคนิคการลดจำนวนสถานะด้วยวิธีแอลฟา-เบตาพรันนิ่ง (Alpha-Beta Pruning) ตลอดจนการประยุกต์ใช้ฟังก์ชันการประเมินสถานะสำหรับเกมที่มีความซับซ้อนสูง

การค้นหาแบบทั่วไปจะพิจารณาเฉพาะการกระทำของตัวแทนเพียงฝ่ายเดียว ในขณะที่การค้นหาแบบปฏิบัติต้องพิจารณาการตอบสนองของคู่ต่อสู้ร่วมด้วย ดังแสดงในรูปที่ 3.1



รูปที่ 3.1 เปรียบเทียบการค้นหาแบบทั่วไปและการค้นหาแบบปฏิบัติ

3.2 แนวคิดพื้นฐานของการค้นหาแบบปฏิบัติ

การค้นหาแบบปฏิบัติ (Adversarial Search) เป็นเทคนิคการค้นหาคำตอบสำหรับปัญหาที่มีผู้เล่นตั้งแต่สองฝ่ายขึ้นไป โดยแต่ละฝ่ายมีวัตถุประสงค์ที่ขัดแย้งกัน การตัดสินใจของผู้เล่นแต่ละฝ่ายจึงส่งผลโดยตรงต่อผลลัพธ์ของอีกฝ่ายหนึ่ง

ในปัญหาประเภทนี้ ผู้เล่นแต่ละฝ่ายจะพยายามเลือกการกระทำที่ให้ผลประโยชน์สูงสุดแก่ตนเอง และในขณะเดียวกันก็พยายามลดโอกาสในการได้รับประโยชน์ของฝ่ายตรงข้าม ดังนั้นระบบปัญญาประดิษฐ์ที่ทำหน้าที่เป็นผู้เล่นจึงจำเป็นต้องสามารถคาดการณ์หรือพยากรณ์การตอบสนองของคู่ต่อสู้ได้

ในทางปฏิบัติ การคาดการณ์ดังกล่าวเกิดขึ้นจากการจำลองลำดับการเดินที่เป็นไปได้ทั้งหมดออกมาในรูปของต้นไม้เกม (Game Tree) แล้วประเมินว่าการเดินในแต่ละแนวทางจะส่งผลต่อผลลัพธ์สุดท้ายอย่างไร

3.2.1 เกมผลรวมศูนย์

ปัญหาส่วนใหญ่ที่ใช้การค้นหาแบบปฏิบัติสามารถอธิบายได้ด้วยแนวคิดของ เกมผลรวมศูนย์ (Zero-Sum Game) ซึ่งหมายถึงเกมที่ผลประโยชน์ของผู้เล่นทั้งสองฝ่ายมีความสัมพันธ์ตรงข้ามกันโดยสิ้นเชิง

หากผู้เล่นฝ่ายหนึ่งได้รับผลประโยชน์เพิ่มขึ้น อีกฝ่ายหนึ่งจะได้รับผลประโยชน์ลดลงในปริมาณที่เท่ากัน

ตัวอย่างเช่น

- หากผู้เล่น A ชนะเกม จะได้รับคะแนน +1
- ผู้เล่น B จะได้รับคะแนน -1

- ดังนั้นผลรวมของคะแนนทั้งสองฝ่ายจะมีค่าเท่ากับศูนย์เสมอ
เกมหมากรุก หมากรอส และทิก-แทค-โท ล้วนเป็นตัวอย่างของเกมผลรวมศูนย์

3.2.2 เกมที่มีข้อมูลสมบูรณ์

เกมที่เหมาะสมสำหรับการประยุกต์ใช้ขั้นตอนวิธีมินิแมกซ์ (เป็นขั้นตอนวิธีการค้นหาแบบปฏิกิริยา จะอธิบายในหัวข้อต่อไป) มักเป็นเกมที่มีข้อมูลสมบูรณ์ (Perfect Information Game) เกมประเภทนี้มีลักษณะดังต่อไปนี้

- 1) ผู้เล่นทุกฝ่ายสามารถมองเห็นสถานะทั้งหมดของเกมได้
- 2) ไม่มีข้อมูลที่ถูกลบซ่อนจากคู่แข่ง
- 3) การเปลี่ยนแปลงของสถานะเกิดจากกฎที่แน่นอน
- 4) ไม่มีองค์ประกอบของความสุ่มเข้ามาเกี่ยวข้อง

ตัวอย่างของเกมที่มีข้อมูลสมบูรณ์ ได้แก่ หมากรุก หมากรอส โกะ และ ทิก-แทค-โท เป็นต้น ในทางตรงกันข้าม เกมบางประเภทมีข้อมูลไม่สมบูรณ์ (Imperfect Information Game) เช่น ไพ่โป๊กเกอร์ ไพ่บาคาร่า ไพ่โป๊กเกอร์ เป็นต้น เนื่องจากผู้เล่นไม่สามารถมองเห็นข้อมูลทั้งหมดของคู่ต่อสู้ได้

3.2.3 คุณสมบัติของปัญหาแบบปฏิกิริยา

การค้นหาแบบปฏิกิริยามีลักษณะที่แตกต่างจากการค้นหาแบบทั่วไปหลายประการ ดังนี้

- มีผู้เล่นมากกว่าหนึ่งฝ่าย
ผู้เล่นแต่ละฝ่ายสามารถส่งผลกระทบต่อผลลัพธ์ของเกมได้
- ผู้เล่นมีเป้าหมายที่ขัดแย้งกัน
สิ่งที่เป็นประโยชน์ต่อฝ่ายหนึ่งอาจเป็นผลเสียต่ออีกฝ่ายหนึ่ง
- จำเป็นต้องพิจารณาการตอบสนองของคู่ต่อสู้
การตัดสินใจที่ดีต้องคำนึงถึงลำดับการเดินในอนาคตของทั้งสองฝ่าย
- จำนวนสถานะมีขนาดใหญ่มาก
เกมที่มีความซับซ้อนสูง เช่น หมากรุก มีจำนวนสถานะที่เป็นไปได้มหาศาล ทำให้ไม่สามารถสำรวจได้ครบทุกสถานะ
- ต้องใช้ฟังก์ชันการประเมิน
ในเกมขนาดใหญ่จำเป็นต้องประมาณค่าความได้เปรียบของสถานะต่าง ๆ แทนการค้นหาจนถึงจุดสิ้นสุดของเกม

เกมที่มีความซับซ้อนสูง เช่น หมากรุก มีจำนวนสถานะที่เป็นไปได้มหาศาล ทำให้ไม่สามารถสำรวจได้ครบทุกสถานะ ต้องใช้ฟังก์ชันการประเมิน ในเกมขนาดใหญ่จำเป็นต้องประมาณค่าความได้เปรียบของสถานะต่าง ๆ แทนการค้นหาจนถึงจุดสิ้นสุดของเกม

ตารางที่ 3.1 เปรียบเทียบการค้นหาแบบทั่วไปและการค้นหาแบบปฏิบัติ

หัวข้อ	การค้นหาแบบทั่วไป	การค้นหาแบบปฏิบัติ
จำนวนผู้เล่น	1 ฝ่าย	2 ฝ่ายขึ้นไป
เป้าหมาย	ค้นหาคำตอบที่ดีที่สุด	เอาชนะคู่ต่อสู้
สภาพแวดล้อม	คงที่	เปลี่ยนแปลงตามการตัดสินใจของคู่แข่ง
ตัวอย่างปัญหา	8 ตัวเลข (8-Puzzle)	หมากรุก หมากรอกอส
ขั้นตอนวิธีที่นิยม	การค้นหาแนวลึก (BFS) การค้นหาแนวกว้าง (DFS) และการค้นหาเอสตาร์ (A*)	มินิแมกซ์
การประเมินผล	ฟังก์ชันราคา (Cost Function)	ฟังก์ชันประเมิน

3.3 การค้นหาแบบเชิงจัดหมู่

การสร้างระบบปัญญาประดิษฐ์สำหรับเกมสามารถทำได้โดยอาศัยแนวคิดของการค้นหาแบบเชิงจัดหมู่ (Combinatorial Search) ซึ่งเป็นกระบวนการสำรวจลำดับการเดินที่เป็นไปได้ทั้งหมดของเกม เพื่อค้นหาการตัดสินใจที่ดีที่สุด

หลักการสำคัญของวิธีนี้ คือ การพิจารณาการกระทำทุกทางเลือกที่สามารถเกิดขึ้นได้ในแต่ละสถานะ จากนั้นจึงสร้างสถานะใหม่ที่เป็นผลลัพธ์ของการกระทำนั้น และดำเนินกระบวนการนี้ต่อไปเรื่อย ๆ จนถึงจุดสิ้นสุดของเกม

ลักษณะดังกล่าวคล้ายกับการค้นหาแบบย้อนกลับ (Backtracking) และการค้นหาแนวลึก (Depth-First Search) ที่ได้ศึกษาไปแล้วในบทที่ผ่านมา

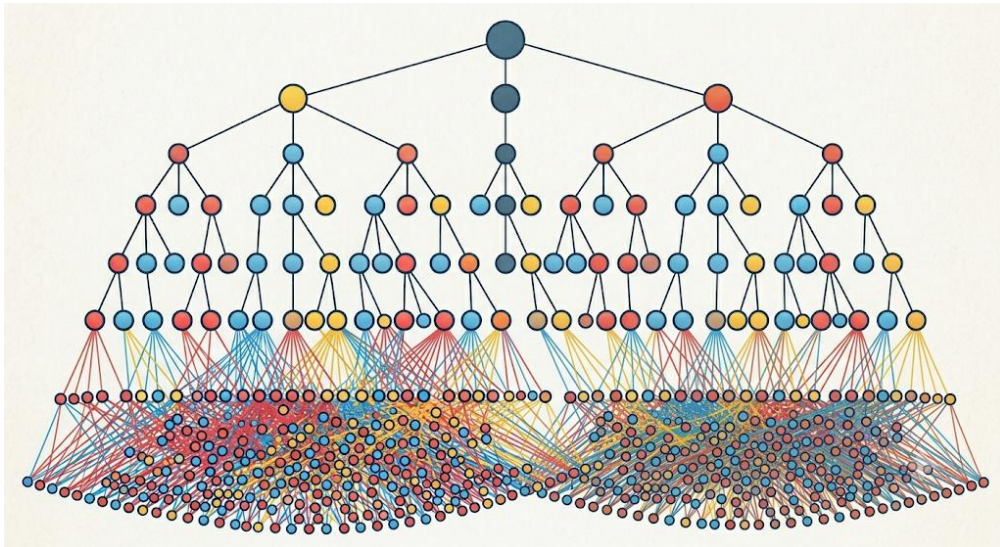
3.3.1 การระเบิดของจำนวนสถานะ (State Space Explosion)

สมมติว่ามีจำนวนทางเลือกเฉลี่ยในแต่ละตาเท่ากับ b และมีความลึกของเกมเท่ากับ d จำนวนสถานะทั้งหมดที่ต้องสำรวจจะมีค่าใกล้เคียงกับ $O(b^d)$ ยิ่งเกมมีความลึกมาก จำนวนสถานะที่ต้องสำรวจก็จะเพิ่มขึ้นแบบเอ็กซ์โพเนนเชียล ตัวอย่างเช่น

- ทิค แทค โท มีจำนวนสถานะประมาณ 10^5
- หมากรอกอส มีจำนวนสถานะประมาณ 10^{20}

- หมากรุก มีจำนวนสถานะประมาณ 10^{120}
- เกมโกะ มีจำนวนสถานะมากกว่า 10^{170}

ดังนั้น การสำรวจทุกสถานะจึงเป็นไปได้สำหรับเกมที่มีความซับซ้อนสูง ดังรูปที่ 3.2



รูปที่ 3.2 การเพิ่มขึ้นของจำนวนสถานะเมื่อความลึกเพิ่มขึ้น

3.3.2 การใช้ฟังก์ชันเรียกตัวเอง

ขั้นตอนวิธีสำหรับเกมส่วนใหญ่นิยมเขียนในลักษณะของฟังก์ชันเรียกตัวเอง (Recursive Function) เนื่องจากโครงสร้างของต้นไม้เกมมีลักษณะเป็นลำดับขั้น (Hierarchical Structure) ซึ่งแต่ละโหนดสามารถแตกแขนงออกไปเป็นสถานะย่อยได้อีกหลายสถานะ แนวคิดดังกล่าวเป็นพื้นฐานสำคัญของขั้นตอนวิธีมินิแมกซ์ ซึ่งจะกล่าวถึงในหัวข้อถัดไป

3.4 องค์ประกอบของเกม

การแก้ปัญหาด้วยวิธีการค้นหาแบบปฏิกิริยาสามารถอธิบายปัญหาในรูปแบบเดียวกับการแก้ปัญหาด้วยการค้นหาในบทก่อนหน้า กล่าวคือ จำเป็นต้องกำหนดองค์ประกอบต่าง ๆ ของปัญหาให้ชัดเจน เพื่อให้ตัวแทนสามารถสร้างต้นไม้เกมและวางแผนการตัดสินใจได้อย่างเป็นระบบ

โดยทั่วไป ปัญหาเกมที่มีข้อมูลสมบูรณ์สามารถอธิบายได้ด้วยองค์ประกอบสำคัญ 6 ส่วน ได้แก่

1. สถานะเริ่มต้น (Initial State)
2. ผู้เล่น (Player)
3. การกระทำ (Action)
4. แบบจำลองการเปลี่ยนสถานะ (Transition Model)

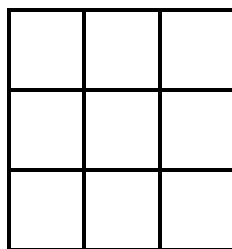
5. สถานะสิ้นสุด (Terminal State)
6. ฟังก์ชันอรรถประโยชน์ (Utility Function)

องค์ประกอบเหล่านี้มีความสำคัญต่อการสร้างต้นไม้เกมและเป็นพื้นฐานของขั้นตอนวิธีมินิแมกซ์ที่จะศึกษาในหัวข้อถัดไป

3.4.1 สถานะเริ่มต้น

สถานะเริ่มต้น (Initial State) หมายถึง สถานะของเกมในขณะที่ยังไม่มีการแข่งขัน ซึ่งประกอบด้วยข้อมูลทั้งหมดที่จำเป็นสำหรับการอธิบายเกม ณ ขณะนั้น

ตัวอย่างเช่น ในเกมทิก-แทค-โท สถานะเริ่มต้นคือกระดานขนาด 3×3 ที่ยังไม่มีการวางสัญลักษณ์ใด ๆ และผู้เล่นฝ่าย X เป็นผู้เริ่มต้นเดินก่อน



รูปที่ 3.3 สถานะเริ่มต้นของเกมทิก-แทค-โท

ในขณะที่ยังไม่มีการวางสัญลักษณ์ สถานะเริ่มต้นจะประกอบด้วยตำแหน่งของหมากทั้ง 16 ตัวที่ถูกจัดวางตามกฎของเกม สถานะเริ่มต้นเป็นจุดเริ่มต้นของการสร้างต้นไม้เกม และเป็นราก (Root Node) ของต้นไม้ทั้งหมด

3.4.2 ผู้เล่น

ผู้เล่น (Player) หมายถึง ฝ่ายที่มีสิทธิ์เลือกการกระทำในสถานะปัจจุบัน

ในเกมสองผู้เล่นโดยทั่วไป จะกำหนดผู้เล่นออกเป็นสองฝ่าย ได้แก่

- MAX (Maximizing Player)
- MIN (Minimizing Player)

ฝ่าย MAX เป็นตัวแทนของปัญญาประดิษฐ์หรือผู้เล่นที่เราต้องการสร้างขึ้น โดยมีเป้าหมายเพื่อให้ได้รับผลตอบแทนสูงสุด

ในขณะที่ฝ่าย MIN แทนคู่ต่อสู้ ซึ่งพยายามลดผลตอบแทนของฝ่าย MAX ให้ต่ำที่สุด

ตัวอย่างเช่น ในเกมทิก-แทค-โท

- X อาจถูกกำหนดให้เป็น MAX
- O อาจถูกกำหนดให้เป็น MIN

ผู้เล่นทั้งสองจะผลัดกันเดินทีละหนึ่งตา จนกระทั่งเกมสิ้นสุดลง

3.4.3 การกระทำ

การกระทำ (Action) หมายถึง การเคลื่อนที่หรือการตัดสินใจที่สามารถทำได้ในสถานะปัจจุบัน เซตของการกระทำที่สามารถเลือกได้เรียกว่า Available Actions หรือ Legal Moves ตัวอย่างเช่น ในเกมทิก-แทค-โท หากมีช่องว่างเหลืออยู่ 5 ช่อง ผู้เล่นสามารถเลือกวางสัญลักษณ์ลงในช่องว่างใดก็ได้ ดังนั้น จำนวนการกระทำที่เป็นไปได้จะเท่ากับ 5 รูปแบบ

ในเกมหมากรุก การกระทำที่เป็นไปได้จะขึ้นอยู่กับกฎการเดินของหมากแต่ละชนิด เช่น

- เดินไปข้างหน้า
- กินหมากฝ่ายตรงข้าม
- เข้าป้อม
- เลื่อนขึ้นเบี้ย

จำนวนการกระทำที่เป็นไปได้ในแต่ละสถานะเรียกว่า Branching Factor ซึ่งมีผลโดยตรงต่อขนาดของต้นไม้เกม

3.4.4 แบบจำลองการเปลี่ยนสถานะ

แบบจำลองการเปลี่ยนสถานะ (Transition Model) หมายถึง กฎที่ใช้กำหนดว่าสถานะใหม่จะเปลี่ยนแปลงไปอย่างไร หลังจากผู้เล่นเลือกการกระทำหนึ่ง ๆ

สามารถเขียนในรูปของฟังก์ชัน

$Result(s,a)$

โดยที่

- s คือสถานะปัจจุบัน
- a คือการกระทำ

ผลลัพธ์ของฟังก์ชันคือสถานะใหม่ที่เกิดขึ้นหลังจากทำการกระทำนั้น ตัวอย่างเช่น สมมติว่าในเกมทิก-แทค-โท ผู้เล่น X เลือกวางสัญลักษณ์ที่ช่องกลาง ดังรูปที่ 3.4

	X	

รูปที่ 3.4 การเปลี่ยนสถานะของเกมทิก-แทค-โท

3.4.5 สถานะสิ้นสุด

สถานะสิ้นสุด (Terminal State) หมายถึง สถานะที่เกมไม่สามารถดำเนินต่อไปได้อีก ตัวอย่างเช่น ใน เกมทิก-แทค-โท เกมจะสิ้นสุดเมื่อ

1. ผู้เล่นฝ่ายใดฝ่ายหนึ่งเรียงสัญลักษณ์ได้ครบสามช่อง หรือ
2. ช่องทั้งเก้าถูกเติมครบทั้งหมดโดยไม่มีผู้ชนะ

ดังรูปที่ 3.5

X	X	X
O	O	

O	X	O
X	O	
O		X

X	O	X
X	X	O
O	X	O

รูปที่ 3.5 สถานะสิ้นสุดของเกมทิก-แทค-โท

3.4.6 ฟังก์ชันให้รางวัล

ฟังก์ชันให้รางวัล (Reward Function) เป็นฟังก์ชันที่ใช้กำหนดค่าผลตอบแทนของสถานะสิ้นสุดแต่ละสถานะในเกมทิก-แทค-โท อาจกำหนดค่าได้ดังนี้

ตารางที่ 3.2 ตัวอย่างการกำหนดค่ารางวัลจากผลลัพธ์สิ้นสุดเกมทิก-แทค-โท

ผลลัพธ์	ค่ารางวัล
X ชนะ	+1
เสมอ	0
MIN ชนะ	-1

หากสถานะสุดท้ายทำให้ฝ่าย X เป็นผู้ชนะ จะได้รับค่าบวกในทางตรงกันข้าม หากฝ่าย O เป็นผู้ชนะ จะได้รับค่าลบ ค่าดังกล่าวจะถูกนำมาใช้ในการคำนวณย้อนกลับภายในขั้นตอนวิธีมินิแมกซ์ เพื่อค้นหาการเดินที่ดีที่สุดสำหรับผู้เล่นฝ่าย X

ตารางที่ 3.3 องค์ประกอบของเกมทิก-แทค-โท

องค์ประกอบ	รายละเอียด
สถานะเริ่มต้น	กระดานว่างขนาด 3×3
ผู้เล่น	X และ O

องค์ประกอบ	รายละเอียด
การเล่น	วางสัญลักษณ์ในช่องว่าง
การเปลี่ยนสถานะ	อัปเดตกระดานหลังการวาง
สถานะสิ้นสุด	ชนะ แพ้ หรือเสมอ
ฟังก์ชันให้รางวัล	+1, 0, -1

3.5 ต้นไม้เกม

การค้นหาแบบปฏิกิริยาอาศัยการแทนลำดับการเดินทั้งหมดในรูปแบบของต้นไม้ ซึ่งเรียกว่า **ต้นไม้เกม (Game Tree)**

ต้นไม้เกมเป็นโครงสร้างข้อมูลแบบลำดับชั้น (Hierarchical Structure) ที่ใช้แสดงสถานะต่าง ๆ ของเกมและความสัมพันธ์ระหว่างสถานะเหล่านั้น โดยแต่ละโหนดจะแทนสถานะหนึ่งของเกม และเส้นเชื่อมระหว่างโหนดจะแทนการกระทำที่ทำให้เกิดการเปลี่ยนแปลงสถานะ

การสร้างต้นไม้เกมช่วยให้ปัญญาประดิษฐ์สามารถพิจารณาลำดับการเดินล่วงหน้าได้หลายตา และประเมินผลลัพธ์ที่อาจเกิดขึ้นจากการตัดสินใจแต่ละทางเลือก

3.5.1 ส่วนประกอบของต้นไม้เกม

ต้นไม้เกมประกอบด้วยองค์ประกอบสำคัญดังต่อไปนี้

โหนดราก

โหนดราก (Root Node) คือสถานะเริ่มต้นของเกม ซึ่งเป็นจุดเริ่มต้นของการค้นหา

โหนดภายใน

โหนดภายใน (Internal Node) คือสถานะที่ยังสามารถดำเนินเกมต่อไปได้

โหนดประเภทนี้สามารถแตกแขนงออกไปยังสถานะลูกได้หลายสถานะ

โหนดใบ

โหนดใบ (Leaf Node) คือสถานะปลายทางของเกม ซึ่งไม่สามารถดำเนินเกมต่อไปได้อีก

สถานะเหล่านี้อาจเป็น ชนะ แพ้ หรือ เสมอ

3.5.2 ระดับความลึกของต้นไม้

ความลึกของต้นไม้ (Depth) หมายถึง จำนวนขั้นของการเดินจากโหนดรากไปยังโหนดปลายทางในเกมสองผู้เล่น แต่ละระดับของต้นไม้แทนการเดินของผู้เล่นสลับกัน

ตัวอย่างเช่น กำหนดให้ผู้เล่นคนแรกในระดับที่ 0 เป็นระดับที่ต้องการเดินแล้วเกิดผลชนะมากที่สุด (Max) และในระดับที่ 1 จะต้องสลับการเดินให้กับคู่ต่อสู้ ทำให้เป้าหมายการเดินในระดับที่ 0 ต้องบังคับให้คู่ต่อสู้มีโอกาสชนะน้อยที่สุด (Min) ทำให้การเดินที่สลับกันไปจะเกิดขึ้นในแต่ละระดับมีลักษณะการเดินดังนี้

ระดับที่ 0 Max โอกาสชนะ

ระดับที่ 1 Min โอกาสชนะ

ระดับที่ 2 Max โอกาสชนะ

ระดับที่ 3 Min โอกาสชนะ

...

ระดับที่ $n \% 2 = 0$ Max โอกาสชนะ

ระดับที่ $n \% 2 = 1$ Min โอกาสชนะ

3.5.3 ปัจจัยการแตกแขนง

ในแต่ละระดับความลึกของต้นไม้จะมีจำนวนทางเลือกเฉลี่ยในแต่ละสถานะเรียกว่า **Branching**

Factor แทนด้วยสัญลักษณ์ b ตัวอย่างเช่น

- Tic-Tac-Toe มีค่า b เฉลี่ยประมาณ 4 ถึง 5
- หมากรอกมีค่า b ประมาณ 3
- เกมโกะมีค่า b มากกว่า 250

หากเกมมีความลึก d จำนวนสถานะที่ต้องสำรวจจะประมาณได้เป็น

$$O(b^d)$$

จะเห็นได้ว่าจำนวนสถานะเพิ่มขึ้นแบบเอ็กซ์โพเนนเชียลอย่างรวดเร็ว ดังนั้น แม้ว่าคอมพิวเตอร์มีความเร็วในการประมวลผลสูง แต่ก็ไม่สามารถสำรวจทุกสถานะได้ครบถ้วนในเกมที่มีความซับซ้อนสูง เช่น หมากรอกหรือเกมโกะ

3.5.4 ตัวอย่างต้นไม้เกมของทิก-แทค-โท

- สมมติว่าผู้เล่น X เริ่มต้นเดินก่อน
 - ในตาแรก X สามารถเลือกวางสัญลักษณ์ได้ทั้งหมด 9 ตำแหน่ง
 - หลังจากนั้น O จะมีทางเลือกเหลือ 8 ตำแหน่ง
 - จากนั้น X จะเหลือทางเลือกอีก 7 ตำแหน่ง
 - กระบวนการดังกล่าวจะดำเนินต่อไปจนกระทั่งเกมสิ้นสุด
- เมื่อนำไปสร้างแผนภาพต้นไม้จะเกิดขึ้นดังรูปที่ 3.6

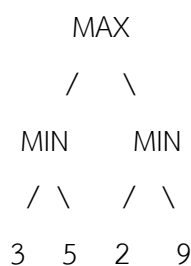
3.6.1 หลักการของมินิแมกซ์

สมมติว่าผู้เล่นฝ่าย MAX เป็นตัวแทนปัญญาประดิษฐ์ที่เราสนใจ และผู้เล่นฝ่าย MIN เป็นคู่ต่อสู้ ในแต่ละสถานะ

- หากเป็นตาของ MAX จะเลือกค่าที่มากที่สุด
- หากเป็นตาของ MIN จะเลือกค่าที่น้อยที่สุด

กระบวนการดังกล่าวจะดำเนินต่อไปจนถึงสถานะปลายทาง จากนั้นจึงส่งค่าผลตอบแทนย้อนกลับ ขึ้นมาตามต้นไม้เกม แนวคิดนี้เรียกว่า **Backward Induction** หรือการวิเคราะห์ย้อนกลับจากผลลัพธ์สุดท้าย

3.6.2 ตัวอย่างการเลือกค่าของ MAX และ MIN



รูปที่ 3.7 การเลือกค่า Max และ Min

การคำนวณมีขั้นตอนดังนี้

- โหนด MIN ทางซ้าย เลือกค่าที่น้อยที่สุด $\min(3,5)=3$
- โหนด MIN ทางขวา เลือกค่าที่น้อยที่สุด $\min(2,9)=2$
- จากนั้นโหนด MAX จะเลือกค่าที่มากที่สุด $\max(3,2)=3$ ดังนั้น ค่าที่ส่งกลับมายังรากของต้นไม้เท่ากับ 3

3.6.3 สมมติฐานของขั้นตอนวิธีมินิแมกซ์

มินิแมกซ์อาศัยสมมติฐานสำคัญหลายประการ ดังนี้

- 1) ผู้เล่นทั้งสองฝ่ายมีเหตุผล

ทั้ง MAX และ MIN จะเลือกการกระทำที่ดีที่สุดสำหรับตนเองเสมอ

- 2) เกมเป็นเกมผลรวมศูนย์

ผลประโยชน์ของฝ่ายหนึ่งเท่ากับความสูญเสียของอีกฝ่ายหนึ่ง

- 3) ไม่มีองค์ประกอบของความสุ่ม

ผลลัพธ์ของการกระทำแต่ละอย่างเป็นไปตามกฎที่แน่นอน

- 4) ผู้เล่นสามารถมองเห็นข้อมูลทั้งหมด

เกมต้องเป็นเกมที่มีข้อมูลสมบูรณ์ (Perfect Information Game)

ตัวอย่างเกมที่เหมาะสมสำหรับมินิแมกซ์ ได้แก่

- ทิค-แทค-โท
- หมากรุก
- หมากฮอส
- โอเทลโล

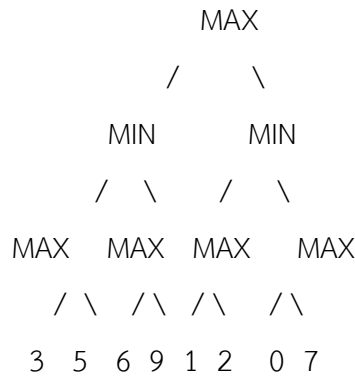
3.6.4 ขั้นตอนการทำงานของมินิแมกซ์

ขั้นตอนการทำงานของมินิแมกซ์สามารถแบ่งออกเป็นสี่ขั้นตอน ดังนี้

- 1) ขั้นตอนที่ 1 สร้างต้นไม้เกม
เริ่มจากสถานะปัจจุบันและแตกแขนงการเดินที่เป็นไปได้ทั้งหมด
- 2) ขั้นตอนที่ 2 ค้นหาจนถึงสถานะปลายทาง
สำรวจต้นไม้เกมจนถึงโหนดใบ ซึ่งเป็นสถานะชนะ แพ้ หรือเสมอ
- 3) ขั้นตอนที่ 3 ประเมินค่าฟังก์ชันรางวัล
กำหนดค่าผลตอบแทนให้กับสถานะปลายทาง
- 4) ขั้นตอนที่ 4 ส่งค่ากลับขึ้นมาตามต้นไม้
โหนด MAX เลือกค่าที่มากที่สุด
โหนด MIN เลือกค่าที่น้อยที่สุด
จนกระทั่งกลับมาถึงโหนดราก
ค่าที่ได้จะเป็นค่าของการเดินที่ดีที่สุด

3.6.5 การคำนวณค่าแบบย้อนกลับ

หลักการสำคัญของมินิแมกซ์คือการเริ่มคำนวณจากโหนดปลายทาง แล้วส่งค่ากลับมายังโหนดระดับบน



รูปที่ 3.8 การส่งค่ากลับจาก Leaf Node

3.6.6 ข้อดีของขั้นตอนวิธีมินิแมกซ์

1. สามารถหาการเดินที่ดีที่สุดได้ หากสำรวจต้นไม้เกมจนถึงสถานะปลายทางทั้งหมด
2. เหมาะสำหรับเกมผลรวมศูนย์ที่มีข้อมูลสมบูรณ์
3. มีหลักการการทำงานที่ชัดเจนและเข้าใจง่าย
4. เป็นพื้นฐานของระบบปัญญาประดิษฐ์สำหรับเกมจำนวนมาก
5. สามารถนำไปประยุกต์ใช้ร่วมกับเทคนิคอื่นได้ เช่น
 - Alpha-Beta Pruning
 - Heuristic Evaluation Function
 - Monte Carlo Tree Search

3.6.7 ข้อจำกัดของขั้นตอนวิธีมินิแมกซ์

แม้ว่ามินิแมกซ์จะสามารถให้คำตอบที่ดีที่สุดได้ แต่ก็มีข้อจำกัดที่สำคัญคือ

3.6.8 จำนวนสถานะมีขนาดใหญ่มาก

เมื่อความลึกของเกมเพิ่มขึ้น จำนวนโหนดจะเพิ่มขึ้นแบบเอ็กซ์โพเนนเชียล $O(b^d)$ โดยที่

- b คือจำนวนทางเลือกเฉลี่ยในแต่ละสถานะ
- d คือความลึกของต้นไม้

ตัวอย่างเช่น ในเกมหมากรุก

- Branching Factor ≈ 35
- ความลึก 8 ชั้น

จำนวนสถานะที่ต้องสำรวจจะมีค่าประมาณ 35^8 หรือมากกว่าสองล้านล้านสถานะ

ทำให้การค้นหาทุกสถานะเป็นไปได้ในทางปฏิบัติ

ดังนั้นจึงจำเป็นต้องมีเทคนิคที่ช่วยลดจำนวนโหนดที่ต้องสำรวจ ซึ่งจะกล่าวถึงในหัวข้อถัดไป คือ การตัดกิ่งด้วยวิธีแอลฟา-เบต้า (Alpha-Beta Pruning) ซึ่งสามารถเพิ่มประสิทธิภาพของมินิแมกซ์ได้อย่างมาก โดยไม่ทำให้ผลลัพธ์สุดท้ายเปลี่ยนแปลงไป

3.7 การตัดกิ่งด้วยวิธีแอลฟา-เบต้า

แม้ว่าขั้นตอนวิธีมินิแมกซ์จะสามารถค้นหาการเดินที่ดีที่สุดได้ แต่ในเกมที่มีจำนวนสถานะขนาดใหญ่ การสำรวจทุกโหนดในต้นไม้เกมอาจต้องใช้เวลาในการประมวลผลสูงมาก ดังนั้นจึงมีการพัฒนาเทคนิคที่ช่วยลดจำนวนโหนดที่ไม่จำเป็นต้องพิจารณา ซึ่งเรียกว่า การตัดกิ่งด้วยวิธีแอลฟา-เบต้า (Alpha-Beta Pruning)

แนวคิดสำคัญของวิธีนี้ คือ การหลีกเลี่ยงการสำรวจโหนดที่ไม่สามารถส่งผลต่อการตัดสินใจสุดท้ายได้ แม้ว่าจะไม่สำรวจโหนดเหล่านั้น ผลลัพธ์ที่ได้จากขั้นตอนวิธียังคงเหมือนกับมินิแมกซ์แบบปกติทุกประการ

กล่าวอีกนัยหนึ่ง การตัดกิ่งด้วยวิธีแอลฟา-เบต้า ไม่ได้เปลี่ยนคำตอบของมินิแมกซ์แต่ช่วยลดจำนวนสถานะที่ต้องสำรวจ ส่งผลให้ใช้เวลาในการประมวลผลน้อยลง

3.7.1 หลักการของการตัดกิ่งแอลฟา-เบต้า

ในระหว่างการค้นหา หากพบว่าโหนดบางส่วนไม่มีโอกาสถูกเลือกอย่างแน่นอน การสำรวจโหนดลูกของโหนดนั้นสามารถหยุดลงได้ทันที กระบวนการดังกล่าวเรียกว่า การตัดกิ่ง (Pruning)

หลักการนี้เปรียบเสมือนการตัดสินใจในชีวิตประจำวัน เช่น สมมติว่านักศึกษาต้องการซื้อคอมพิวเตอร์เครื่องใหม่ โดยมีงบประมาณไม่เกิน 25,000 บาท หากร้านแรกมีคอมพิวเตอร์ที่มีคุณสมบัติตรงตามความต้องการในราคา 22,000 บาท แล้วเดินไปดูร้านถัดไปพบว่าราคาสูงถึง 35,000 บาท นักศึกษาอาจไม่จำเป็นต้องพิจารณารายละเอียดอื่นเพิ่มเติม เนื่องจากสินค้าดังกล่าวไม่สามารถเป็นตัวเลือกที่ดีกว่าได้

ในลักษณะเดียวกันการตัดกิ่งด้วยวิธีแอลฟา-เบต้าจะตัดกิ่งของต้นไม้เกมที่ไม่สามารถให้ผลลัพธ์ที่ดีกว่าโหนดที่พิจารณาไว้ก่อนหน้านี้

3.7.2 การทำงานของแอลฟา-เบต้า

การทำงานของขั้นตอนวิธีการทำงานของขั้นตอนวิธีนี้อาศัยค่าพารามิเตอร์สองค่า ได้แก่

ค่าแอลฟา (Alpha) แทนด้วยสัญลักษณ์ α หมายถึง ค่าที่ดีที่สุดที่ฝ่าย MAX สามารถรับประกันได้ในขณะนั้น หรืออาจกล่าวได้ว่า ค่าแอลฟาเป็นขอบเขตล่าง (Lower Bound) ของผลลัพธ์ที่ฝ่าย MAX ยอมรับได้

ในช่วงเริ่มต้น $\alpha = -\infty$

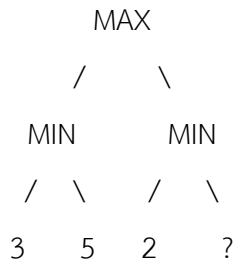
ค่าเบตา (Beta) แทนด้วยสัญลักษณ์ β หมายถึง ค่าที่ดีที่สุดที่ฝ่าย MIN สามารถรับประกันได้ในขณะนั้น หรือ เป็นขอบเขตบน (Upper Bound) ที่ฝ่าย MIN ยอมให้ฝ่าย MAX ได้รับ

ในช่วงเริ่มต้น $\beta = +\infty$

3.7.3 เงื่อนไขการตัดกิ่ง

ระหว่างการค้นหา หากพบว่า $\alpha \geq \beta$ สามารถหยุดการสำรวจโหนดที่เหลือได้ทันที เนื่องจากไม่ว่าค่าที่เหลือจะเป็นเท่าใด ก็ไม่สามารถเปลี่ยนแปลงการตัดสินใจของผู้เล่นได้อีก เงื่อนไขดังกล่าวเรียกว่า Alpha-Beta Cutoff

การตัดกิ่งที่โหนด MIN



รูปที่ 3.9 หลักการทำงานของ การตัดกิ่งด้วยวิธีแอลฟา-บีตา

เริ่มจากกิ่งทางซ้าย โหนด MIN เลือกค่าต่ำสุด $\min(3,5)=3$

ดังนั้น ค่า Alpha ของโหนดรากจะกลายเป็น $\alpha = 3$

จากนั้นพิจารณากิ่งทางขวา เมื่อเข้าสู่โหนด MIN พบค่าแรกเท่ากับ 2 ดังนั้น $\beta = 2$

เนื่องจาก $\alpha = 3$ และ $\beta = 2$ จึงได้ว่า $\alpha > \beta$ ทำให้สามารถหยุดการสำรวจค่าที่เหลือทางด้านขวาได้ทันที

ดังนั้นโหนดที่มีเครื่องหมาย ? จึงไม่จำเป็นต้องถูกคำนวณ

อธิบายเพิ่มเติม

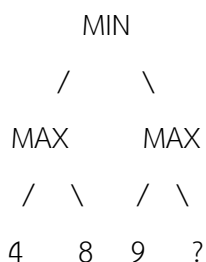
กรณีที่ 1 $\min(2,?)$ จะเท่ากับ 2 หาก ? มีค่ามากกว่า 2 ทำให้ $\max(3,2) = 3$

กรณีที่ 2 $\min(2,?)$ จะเท่ากับ 1 หาก ? มีค่าน้อยกว่า 2 ทำให้ $\max(3,?) = 3$

จะพบว่าไม่ว่ากรณีใด ค่าของ $\max(3, \min(2 | ?)) = 3$ เสมอ

การตัดกิ่งที่โหนด MAX

การตัดกิ่งไม่ได้เกิดขึ้นเฉพาะในโหนด MIN เท่านั้น แต่สามารถเกิดขึ้นในโหนด MAX ได้เช่นเดียวกัน พิจารณาตัวอย่างดังต่อไปนี้



รูปที่ 3.10 การตัดกิ่งที่โหนด MAX

$\beta = \max(4,8)=8$ และ $\alpha = 9$ แต่ $\alpha > \beta$ จึงสามารถหยุดการสำรวจโหนดที่เหลือได้ โหนดที่มีเครื่องหมาย ? จะถูกตัดออกทันที

ลำดับการสำรวจมีผลต่อประสิทธิภาพ ประสิทธิภาพของการตัดกิ่งด้วยวิธีแอลฟา-เบต้าขึ้นอยู่กับลำดับการสำรวจโหนด หากสำรวจโหนดที่มีค่าดีที่สุดก่อน จะสามารถตัดกิ่งได้จำนวนมาก ในทางกลับกัน หากสำรวจโหนดที่ไม่เหมาะสมก่อน จำนวนโหนดที่ถูกตัดออกจะลดลง ดังนั้น ลำดับการเดินจึงมีผลอย่างมากต่อประสิทธิภาพของขั้นตอนวิธี

3.7.4 การวิเคราะห์ความซับซ้อน

มินิแมกซ์ปกติมีความซับซ้อนเชิงเวลาเท่ากับ $O(b^d)$ โดยที่

- b คือ Branching Factor
- d คือความลึกของต้นไม้เกม

เมื่อใช้การตัดกิ่งด้วยวิธีแอลฟา-เบต้าและมีการจัดลำดับการเดินอย่างเหมาะสม ความซับซ้อนอาจลดลงเหลือประมาณ $O(b^{d/2})$ กล่าวคือ สามารถค้นหาได้ลึกขึ้นเป็นสองเท่า โดยใช้เวลาใกล้เคียงเดิม

3.7.5 ข้อดีของการตัดกิ่งด้วยวิธีแอลฟา-เบต้า

1. ให้ผลลัพธ์เหมือนกับมินิแมกซ์ทุกประการ
2. ลดจำนวนโหนดที่ต้องสำรวจ
3. ลดเวลาในการประมวลผล
4. สามารถค้นหาได้ลึกมากขึ้น
5. เหมาะสำหรับเกมที่มีจำนวนสถานะขนาดใหญ่
6. เป็นพื้นฐานของโปรแกรมหมากรุกและเกมกระดานสมัยใหม่

แม้ว่าการตัดกิ่งด้วยวิธีแอลฟา-เบต้าจะช่วยลดจำนวนสถานะที่ต้องสำรวจได้อย่างมาก แต่ในเกมที่มีความซับซ้อนสูง เช่น หมากรุกหรือเกมโกะ จำนวนสถานะที่เป็นไปได้ก็ยังมีขนาดใหญ่เกินกว่าจะค้นหาจนถึงจุดสิ้นสุดของเกมได้ทั้งหมด

ดังนั้น ในทางปฏิบัติ จึงจำเป็นต้องหยุดการค้นหาที่ระดับความลึกหนึ่ง แล้วใช้ฟังก์ชันการประเมินสถานะเพื่อประมาณว่าฝ่ายใดอยู่ในสถานะที่ได้เปรียบ ซึ่งจะกล่าวถึงในหัวข้อถัดไป

3.8 ฟังก์ชันการประเมินสถานะ

จากหัวข้อที่ผ่านมา จะเห็นได้ว่าขั้นตอนวิธีมินิแมกซ์และแอลฟา-เบตาพูนนิ่งต้องอาศัยการสำรวจต้นไม้เกมจนกระทั่งถึงสถานะปลายทาง เพื่อให้สามารถทราบผลลัพธ์ที่แน่นอนของเกมว่าฝ่ายใดเป็นผู้ชนะหรือผู้แพ้ อย่างไรก็ตาม ในเกมที่มีความซับซ้อนสูง เช่น หมากรุกหรือเกมโกะ จำนวนสถานะที่เป็นไปได้มีขนาดใหญ่มากจนไม่สามารถสำรวจต้นไม้เกมทั้งหมดได้ภายในเวลาที่เหมาะสม

ดังนั้น ในทางปฏิบัติ โปรแกรมปัญญาประดิษฐ์จึงมักหยุดการค้นหาที่ระดับความลึกหนึ่ง แล้วใช้วิธีประมาณค่าความได้เปรียบของสถานะ ณ ขณะนั้นแทนการค้นหาจนถึงจุดสิ้นสุดของเกม วิธีการดังกล่าวเรียกว่า ฟังก์ชันการประเมินสถานะ (Evaluation Function) หรือ Heuristic Evaluation Function ฟังก์ชันการประเมินสถานะเป็นองค์ประกอบสำคัญที่ทำให้ระบบปัญญาประดิษฐ์สามารถตัดสินใจได้อย่างมีประสิทธิภาพ แม้ว่าจะไม่สามารถวิเคราะห์ต้นไม้เกมทั้งหมดได้ก็ตาม

3.8.1 แนวคิดของฟังก์ชันการประเมิน

ฟังก์ชันการประเมินเป็นฟังก์ชันที่ใช้ประมาณค่าความได้เปรียบของสถานะหนึ่ง ๆ โดยพิจารณาจากลักษณะหรือคุณสมบัติที่สำคัญของเกม

ค่าที่ได้จากฟังก์ชันการประเมินจะถูกใช้แทนค่า Utility ของสถานะปลายทางในกรณีที่การค้นหาถูกหยุดก่อนถึงจุดสิ้นสุดของเกม

โดยทั่วไปสามารถตีความค่าที่ได้ดังนี้

- ค่าบวก หมายถึง ฝ่าย MAX มีความได้เปรียบ
- ค่าลบ หมายถึง ฝ่าย MIN มีความได้เปรียบ
- ค่าใกล้ศูนย์ หมายถึง ทั้งสองฝ่ายมีโอกาสใกล้เคียงกัน

ตัวอย่างเช่น

ค่าการประเมิน ความหมาย

+100	MAX ได้เปรียบมาก
+20	MAX ได้เปรียบเล็กน้อย
0	สูสีกัน
-20	MIN ได้เปรียบเล็กน้อย
-100	MIN ได้เปรียบมาก

ค่าดังกล่าวไม่ได้แสดงผลแพ้หรือชนะโดยตรง แต่เป็นเพียงการประมาณว่าฝ่ายใดมีโอกาสชนะมากกว่า

3.8.2 การหยุดการค้นหา

ในเกมที่มีจำนวนสถานะขนาดใหญ่ การค้นหาจะถูกหยุดที่ระดับความลึกที่กำหนดไว้ล่วงหน้า ซึ่งเรียกว่า Cut-off Depth หลังจากนั้นจะใช้ฟังก์ชันการประเมินแทนการค้นหาต่อไปจนถึงสถานะปลายทาง

3.8.3 คุณสมบัติของฟังก์ชันการประเมินที่ดี

- 1) สามารถคำนวณได้รวดเร็ว
เนื่องจากการเรียกใช้งานจำนวนมากระหว่างการค้นหา หากฟังก์ชันมีความซับซ้อนมากเกินไป อาจทำให้เวลาในการประมวลผลเพิ่มขึ้นอย่างมาก
- 2) สามารถสะท้อนความได้เปรียบของเกมได้อย่างเหมาะสม
ค่าที่ได้ควรมีความสัมพันธ์กับโอกาสในการชนะจริง
- 3) มีความสอดคล้องกับฟังก์ชันให้รางวัล
หากสถานะใดมีโอกาสชนะสูง ค่าการประเมินควรมีค่าสูงตามไปด้วย
- 4) สามารถเปรียบเทียบสถานะต่าง ๆ ได้
ค่าที่ได้ควรช่วยให้ระบบสามารถเลือกสถานะที่ดีกว่าได้อย่างชัดเจน

3.8.4 ฟังก์ชันการประเมินเชิงเส้น

วิธีที่นิยมใช้ในการออกแบบฟังก์ชันการประเมิน คือ การรวมคะแนนจากคุณลักษณะต่าง ๆ ของเกมเข้าด้วยกันในรูปแบบของสมการเชิงเส้น

สามารถเขียนได้ดังนี้

$$Score = w_1f_1 + w_2f_2 + \dots + w_nf_n$$

โดยที่

- f_i แทนคุณลักษณะ (Feature)
 - w_i แทนค่าน้ำหนัก (Weight)
ค่าน้ำหนักที่มีค่ามาก หมายถึงคุณลักษณะนั้นมีความสำคัญต่อการตัดสินใจมาก
ตัวอย่างเช่น หากกำหนดให้
 - จำนวนตัวหมากมีความสำคัญมาก
 - ตำแหน่งของหมากมีความสำคัญปานกลาง
- ก็สามารถกำหนดค่าน้ำหนักของจำนวนตัวหมากให้สูงกว่าตำแหน่งของหมากได้

3.8.5 ตัวอย่างฟังก์ชันการประเมินอย่างง่าย

พิจารณาเกมที่มีคุณลักษณะสองตัว ได้แก่

1. จำนวนตัวหมากที่เหลืออยู่
 2. จำนวนทางเลือกในการเดิน
- สมมติว่ากำหนดสมการเป็น

$$Score = 5(Piece) + 2(Mobility)$$

หากสถานะหนึ่งมี

- จำนวนตัวหมาก 6 ตัว
 - จำนวนทางเลือกในการเดิน 4 รูปแบบ
- จะได้ว่า

$$\begin{aligned} Score &= (5 \times 6) + (2 \times 4) \\ &= 30 + 8 \\ &= 38 \end{aligned}$$

ในขณะที่อีกสถานะหนึ่งอาจมีคะแนนเพียง 25 ดังนั้น ปัญญาประดิษฐ์จะเลือกสถานะที่มีคะแนน 38 เพราะถือว่าเป็นสถานะที่มีความได้เปรียบมากกว่า

3.8.6 ความแตกต่างระหว่างฟังก์ชันให้รางวัลและฟังก์ชันประเมิน

ตารางที่ 3.4 เปรียบเทียบฟังก์ชันให้รางวัลและฟังก์ชันประเมิน

หัวข้อ	ฟังก์ชันให้รางวัล	ฟังก์ชันประเมิน
ใช้กับ	สถานะปลายทาง	สถานะระหว่างทาง
ผลลัพธ์	ค่าที่แน่นอน	ค่าประมาณ
ความถูกต้อง	ถูกต้องสมบูรณ์	อาจคลาดเคลื่อน
เวลาในการคำนวณ	สูง	ต่ำ
ตัวอย่างค่า	+1, 0, -1	+80, -45, +12

3.8.7 ความสัมพันธ์ระหว่างมินิแมกซ์และฟังก์ชันการประเมิน

ในกรณีที่ต้นไม้เกมมีขนาดเล็ก เช่น เกมทิก-แทค-โท สามารถค้นหาไปจนถึงสถานะปลายทางได้โดยตรง แต่สำหรับเกมที่มีความซับซ้อนสูง การทำงานของมินิแมกซ์จะมีลักษณะดังนี้

- 1) สร้างต้นไม้เกม
- 2) ใช้การตัดกิ่งด้วยวิธีแอลฟา-บีตา เพื่อลดจำนวนโหนด
- 3) หยุดการค้นหาที่ระดับความลึกที่กำหนด
- 4) ใช้ฟังก์ชันการประเมิน ประเมินค่าโหนดที่อยู่ระดับล่างสุด
- 5) ส่งค่ากลับขึ้นมาตามหลักการของมินิแมกซ์

3.9 ตัวอย่างการออกแบบฟังก์ชันการประเมิน

การออกแบบฟังก์ชันการประเมินขึ้นอยู่กับลักษณะของเกมแต่ละประเภท เนื่องจากแต่ละเกมมีปัจจัยที่ส่งผลต่อความได้เปรียบแตกต่างกัน โดยทั่วไป กระบวนการออกแบบฟังก์ชันการประเมินประกอบด้วยขั้นตอนดังต่อไปนี้

- 1) เลือกคุณลักษณะที่สำคัญของเกม
- 2) กำหนดค่าน้ำหนักของแต่ละคุณลักษณะ
- 3) รวมคะแนนทั้งหมดเข้าด้วยกัน
- 4) ใช้ค่าที่ได้ในการเปรียบเทียบสถานะต่าง ๆ

3.9.1 การประเมินจำนวนตัวหมาก

จำนวนตัวหมากที่เหลืออยู่เป็นปัจจัยพื้นฐานที่นิยมใช้ในเกมกระดานหลายชนิด ตัวอย่างเช่น ในเกมหมากฮอส

- ตัวธรรมดา มีค่า 1 คะแนน
- ฮอส มีค่า 3 คะแนน
- สมมติว่าฝ่าย MAX มี ตัวธรรมดา 5 ตัว ฮอส 1 ตัว
- ฝ่าย MIN มี ตัวธรรมดา 4 ตัว
- คะแนนรวมของสถานะดังกล่าวเท่ากับ $5(1)+1(3)-4(1) = 4$
- ดังนั้น สถานะนี้ถือว่าฝ่าย MAX มีความได้เปรียบ

3.9.2 ตำแหน่งบนกระดาน

ในหลายเกม ตำแหน่งของตัวหมากมีความสำคัญไม่น้อยไปกว่าจำนวนของตัวหมาก

ตัวอย่างเช่น ในหมากฮอส ตัวหมากที่อยู่บริเวณกลางกระดานมักมีโอกาสเคลื่อนที่และโจมตีได้หลายทิศทางมากกว่าตัวหมากที่อยู่บริเวณขอบกระดาน

ดังนั้น จึงอาจกำหนดคะแนนเพิ่มเติมให้แก่ตัวหมากที่อยู่ในตำแหน่งสำคัญ

3.9.3 ความคล่องตัวในการเดิน

ความคล่องตัว (Mobility) หมายถึง จำนวนทางเลือกที่สามารถเดินได้ในสถานะปัจจุบัน หากผู้เล่นมีจำนวนทางเลือกมาก ย่อมมีความยืดหยุ่นในการวางแผนมากกว่า ในทางตรงกันข้าม หากมีทางเลือกเพียงเล็กน้อย ผู้เล่นอาจตกอยู่ในสถานการณ์ที่เสียเปรียบ

ดังนั้น จำนวนทางเลือกในการเดินจึงมักถูกนำมาใช้เป็นคุณลักษณะหนึ่งในฟังก์ชันการประเมิน

3.9.4 การคุกคามและการป้องกัน

ปัจจัยอีกประการหนึ่งที่นิยมใช้ในการประเมินสถานะ คือ ความเสี่ยงที่ตัวหมากจะถูกโจมตีหรือถูกกิน ในตาถัดไป ตัวอย่างเช่น ตัวหมากที่กำลังจะถูกกิน อาจถูกหักคะแนน ตัวหมากที่ได้รับการป้องกันจากหมากตัวอื่น อาจได้รับคะแนนเพิ่ม

การประเมินลักษณะดังกล่าวช่วยให้ระบบปัญญาประดิษฐ์สามารถตัดสินใจได้ใกล้เคียงกับการคิดของมนุษย์มากขึ้น

ตารางที่ 3.5 ปัจจัยที่นิยมใช้ในการออกแบบฟังก์ชันการประเมิน

ปัจจัย	ความหมาย
Piece Advantage	จำนวนตัวหมาก
Positioning	ตำแหน่งบนกระดาน
Mobility	จำนวนทางเลือกในการเดิน
Threat	ความเสี่ยงที่จะถูกโจมตี
Defense	การป้องกันตัวหมาก
Control	การควบคุมพื้นที่สำคัญ

3.10 ตัวอย่างการประยุกต์ใช้ขั้นตอนวิธีมินิแมกซ์กับเกมหยิบเหรียญ

เพื่อให้เห็นภาพการทำงานของขั้นตอนวิธีมินิแมกซ์อย่างชัดเจน ในหัวข้อนี้จะพิจารณาตัวอย่างเกมอย่างง่ายที่เรียกว่า เกมหยิบเหรียญ (Coin Game) ซึ่งเป็นปัญหาที่มีจำนวนสถานะไม่มากนัก ทำให้สามารถสร้างต้นไม้เกมและวิเคราะห์กระบวนการตัดสินใจได้อย่างละเอียด

แม้ว่าเกมดังกล่าวจะมีความเรียบง่าย แต่แนวคิดที่ใช้ในการแก้ปัญหาสามารถนำไปประยุกต์ใช้กับเกมที่มีความซับซ้อนมากขึ้น เช่น หมากรุก หมากฮอส หรือเกมกระดานประเภทอื่น ๆ ได้

3.10.1 กติกาของเกม

สมมติว่ามีเหรียญจำนวน 5 เหรียญวางอยู่บนโต๊ะ

ผู้เล่นสองฝ่ายผลัดกันหยิบเหรียญ โดยแต่ละตาสามารถเลือกหยิบได้

- ครั้งละ 1 เหรียญ
- ครั้งละ 2 เหรียญ

ผู้เล่นที่หยิบเหรียญเหรียญสุดท้ายจะเป็นผู้ชนะ

สมมติให้

- ผู้เล่นคนแรกเป็นฝ่าย MAX
- ผู้เล่นคนที่สองเป็นฝ่าย MIN

คำถามคือ

หากทั้งสองฝ่ายเล่นอย่างดีที่สุด ผู้เล่นฝ่าย MAX ควรหยิบเหรียญจำนวนเท่าใดในการเดินครั้งแรก

○ ○ ○ ○ ○

รูปที่ 3.11 สถานะเริ่มต้นของเกมหยิบเหรียญ

จากเกมเริ่มต้นด้วยเหรียญจำนวน 5 เหรียญ และผู้เล่นทั้งสองฝ่ายสามารถหยิบได้ครั้งละ 1 หรือ 2 เหรียญ

3.10.2 การแทนสถานะของเกม

ในการแก้ปัญหาด้วยวิธีการค้นหา จำเป็นต้องกำหนดองค์ประกอบของปัญหาให้ชัดเจน

- **สถานะ (State)**

สถานะของเกมสามารถแทนด้วย (coins_remaining, current_player) โดยที่

- coins_remaining คือ จำนวนเหรียญที่เหลืออยู่
- current_player คือ ผู้เล่นที่เป็นฝ่ายเดินในขณะนั้น

ตัวอย่างเช่น (5, MAX) หมายถึง มีเหรียญเหลืออยู่ 5 เหรียญ และเป็นตาของฝ่าย MAX

- **สถานะเริ่มต้น**

สถานะเริ่มต้นของปัญหาคือ (5, MAX)

- **การกระทำ**

ผู้เล่นสามารถเลือกการกระทำได้สองแบบ

1. หยิบเหรียญ 1 เหรียญ
2. หยิบเหรียญ 2 เหรียญ

- **สถานะสิ้นสุด**

เกมจะสิ้นสุดเมื่อ $\text{coins_remaining} = 0$ ผู้เล่นที่เป็นผู้หยิบเหรียญสุดท้ายจะเป็นผู้ชนะ

- ฟังก์ชันอรรถประโยชน์

กำหนดให้

ผลลัพธ์	คะแนน
MAX ชนะ	+1
MIN ชนะ	-1

3.10.3 การสร้างต้นไม้เกม

เริ่มต้นจากสถานะ (5, MAX) ฝ่าย MAX สามารถเลือกได้สองทาง

- หยิบ 1 เหรียญ เหลือ 4 เหรียญ
- หยิบ 2 เหรียญ เหลือ 3 เหรียญ

จากนั้นจะเป็นตาของฝ่าย MIN

ต้นไม้เกมของปัญหาหยิบเหรียญ



แต่ละโหนดจะถูกแตกแขนงต่อไปจนกระทั่งจำนวนเหรียญเหลือเท่ากับศูนย์

ในการคิดโจทย์ปัญหาข้อนี้ จะใช้วิธีการคิดแบบย้อนกลับ นั่นคือการคิดจากจุดสิ้นสุดเกมส์ดังหัวข้อที่
จะกล่าวต่อไป

3.10.4 การวิเคราะห์สถานะที่มีเหรียญเหลือ 1 เหรียญ

พิจารณาสถานะที่เหลือเหรียญเพียง 1 เหรียญ ผู้เล่นที่ถึงตาเดินสามารถหยิบเหรียญทั้งหมดได้ทันที
ดังนั้น สถานะที่มีเหรียญเหลือ 1 เหรียญจึงเป็นสถานะที่ได้เปรียบสำหรับผู้เล่นที่กำลังเดิน และจะได้รับค่า +1
หากผู้เล่นคนนั้นเป็นฝ่าย MAX หรือ -1 หากผู้เล่นคนนั้นเป็นฝ่าย MIN

3.10.5 การวิเคราะห์สถานะที่มีเหรียญเหลือ 2 เหรียญ

เมื่อเหลือเหรียญ 2 เหรียญ ผู้เล่นที่ถึงตาสามารถ

- หยิบ 1 เหรียญ
- หรือหยิบทั้ง 2 เหรียญ

ดังนั้น ผู้เล่นที่กำลังเดินสามารถทำให้เกมจบลงและเป็นผู้ชนะได้ทันที จึงถือว่าเป็นสถานะที่ได้เปรียบสำหรับผู้เล่นที่กำลังเดินเช่นเดียวกัน ดังนั้นสถานการณ์เหลือ 2 เหรียญไม่แตกต่างจากการเหลือ 1 เหรียญ

3.10.6 การวิเคราะห์สถานะที่มีเหรียญเหลือ 3 เหรียญ

พิจารณาสถานะที่มีเหรียญเหลืออยู่ 3 เหรียญ ผู้เล่นสามารถเลือกได้สองทาง

- กรณีที่หนึ่ง

หยิบ 1 เหรียญ จะเหลือ 2 เหรียญ ซึ่งเป็นสถานะที่ผู้เล่นฝ่ายตรงข้ามสามารถชนะได้ทันที

- กรณีที่สอง

หยิบ 2 เหรียญ จะเหลือ 1 เหรียญ ซึ่งฝ่ายตรงข้ามก็สามารถหยิบเหรียญสุดท้ายและชนะได้เช่นเดียวกัน

ดังนั้น ไม่ว่าผู้เล่นจะเลือกกรณีที่หนึ่งหรือสอง ฝ่ายตรงข้ามก็สามารถชนะได้เสมอ จึงกล่าวได้ว่าสถานะที่มีเหรียญเหลือ 3 เหรียญเป็นสถานะแพ้สำหรับผู้เล่นที่กำลังเดิน

วิเคราะห์สถานะที่มีเหรียญเหลือ 3 เหรียญ

3
/ \
2 1
ชนะ ชนะ
สำหรับคู่ต่อสู้

3.10.7 การวิเคราะห์สถานะที่มีเหรียญเหลือ 4 เหรียญ

พิจารณาสถานะที่เหลือเหรียญ 4 เหรียญ

ผู้เล่นสามารถเลือกได้ดังนี้

กรณีที่หนึ่ง หยิบ 1 เหรียญ จะเหลือเหรียญ 3 เหรียญ ซึ่งเป็นสถานะแพ้ของฝ่ายตรงข้าม

กรณีที่สอง หยิบ 2 เหรียญ จะเหลือเหรียญ 2 เหรียญ ซึ่งเป็นสถานะชนะของฝ่ายตรงข้าม

ดังนั้น ผู้เล่นสามารถเลือกโอกาสที่จะชนะได้นั้นคือ เลือกกรณีแรก ทำให้คู่ต่อสู้ต้องเผชิญกับสถานะที่มีเหรียญเหลือ 3 เหรียญ จึงถือว่าสถานะที่มีเหรียญเหลือ 4 เหรียญเป็นสถานะชนะ

3.10.8 การวิเคราะห์สถานะเริ่มต้น

เมื่อเริ่มต้นเกมมีเหรียญทั้งหมด 5 เหรียญ ฝ่าย MAX มีทางเลือกสองทาง

กรณีที่หนึ่ง หยิบ 1 เหรียญ เหลือเหรียญ 4 เหรียญ ซึ่งเป็นสถานะชนะสำหรับฝ่าย MIN ดังนั้น ทางเลือกนี้ไม่เหมาะสม

กรณีที่สอง หยิบ 2 เหรียญ เหลือเหรียญ 3 เหรียญ ซึ่งเป็นสถานะแพ้สำหรับฝ่าย MIN ดังนั้น ฝ่าย MAX ควรเลือกหยิบ 2 เหรียญ เพื่อบังคับให้ฝ่าย MIN อยู่ในสถานะเสียเปรียบ

ต้นไม้เกมของปัญหาหยิบเหรียญจำนวน 5 เหรียญ



ดังนั้น การเดินที่ดีที่สุดของฝ่าย MAX คือ หยิบเหรียญจำนวน 2 เหรียญในการเดินครั้งแรก

- กรณี 6 เหรียญนิสิตคิดว่าควรหยิบกี่เหรียญเพื่อให้เกิดโอกาสชนะมากที่สุด เพราะเหตุใด?
- กรณี n เหรียญนิสิตคิดว่าควรหยิบกี่เหรียญเพื่อให้เกิดโอกาสชนะมากที่สุด เพราะเหตุใด?

3.10.9 การมองปัญหาในรูปแบบของมินิแมกซ์

เมื่อพิจารณาในมุมมองของขั้นตอนวิธีมินิแมกซ์

- ฝ่าย MAX พยายามเลือกทางที่ทำให้ตนเองมีโอกาสชนะมากที่สุด
- ฝ่าย MIN พยายามเลือกทางที่ทำให้ฝ่าย MAX เสียเปรียบมากที่สุด

การส่งค่าจากสถานะปลายทางย้อนกลับขึ้นมาตามต้นไม้เกม จะทำให้สามารถระบุได้ว่าการเดินใดเป็นการเดินที่เหมาะสมที่สุด

แม้ว่าปัญหาเกมหยิบเหรียญจะมีขนาดเล็ก แต่หลักการเดียวกันนี้สามารถนำไปประยุกต์ใช้กับเกมที่มีความซับซ้อนสูงได้ เพียงแต่ในเกมขนาดใหญ่จำเป็นต้องใช้การตัดกิ่งด้วยวิธีแอลฟา-บีตาและฟังก์ชันการประเมินสถานะเข้าช่วย เพื่อให้สามารถคำนวณได้ภายในเวลาที่เหมาะสม

3.11 บทสรุป

การแก้ปัญหาด้วยวิธีการค้นหาแบบปฏิบัติเป็นแนวทางสำคัญของปัญญาประดิษฐ์ที่ใช้สำหรับปัญหาซึ่งมีผู้เล่นมากกว่าหนึ่งฝ่าย และผู้เล่นแต่ละฝ่ายมีวัตถุประสงค์ที่ขัดแย้งกัน โดยเฉพาะอย่างยิ่งเกมประเภทผลรวมศูนย์ที่ความสำเร็จของผู้เล่นฝ่ายหนึ่งหมายถึงความล้มเหลวของอีกฝ่ายหนึ่ง ในการตัดสินใจ ตัวแทนจึงจำเป็นต้องพิจารณาไม่เพียงแต่ผลลัพธ์ของการกระทำของตนเองเท่านั้น แต่ยังต้องคาดการณ์การตอบสนองที่เป็นไปได้ของคู่ต่อสู้อีกด้วย

การแทนปัญหาในรูปของต้นไม้เกมช่วยให้สามารถอธิบายลำดับการเดินที่อาจเกิดขึ้นทั้งหมดได้อย่างเป็นระบบ โดยแต่ละโหนดแทนสถานะหนึ่งของเกม และแต่ละเส้นเชื่อมแทนการกระทำที่นำไปสู่สถานะถัดไป แนวคิดดังกล่าวเป็นพื้นฐานสำคัญของขั้นตอนวิธีมินิแมกซ์ ซึ่งใช้หลักการเลือกค่ามากที่สุดสำหรับผู้เล่นฝ่าย MAX และเลือกค่าน้อยที่สุดสำหรับผู้เล่นฝ่าย MIN ภายใต้สมมติฐานว่าผู้เล่นทั้งสองฝ่ายต่างมีเหตุผลและพยายามเลือกการกระทำที่ดีที่สุดสำหรับตนเอง

แม้ว่าขั้นตอนวิธีมินิแมกซ์จะสามารถค้นหาการเดินที่เหมาะสมที่สุดได้ แต่จำนวนสถานะของเกมจะเพิ่มขึ้นอย่างรวดเร็วตามความลึกของต้นไม้เกม ส่งผลให้ต้องใช้เวลาในการประมวลผลและหน่วยความจำจำนวนมาก ดังนั้นจึงมีการพัฒนาเทคนิคแอลฟา-เบตาพRUNNING ซึ่งช่วยลดจำนวนโหนดที่ไม่จำเป็นต้องสำรวจโดยไม่ส่งผลกระทบต่อผลลัพธ์สุดท้ายของขั้นตอนวิธีมินิแมกซ์ ทำให้สามารถค้นหาคำตอบได้รวดเร็วและมีประสิทธิภาพมากยิ่งขึ้น

สำหรับเกมที่มีความซับซ้อนสูง เช่น หมากรุกหรือเกมโกะ การสำรวจต้นไม้เกมจนถึงสถานะปลายทางทั้งหมดไม่สามารถทำได้ในทางปฏิบัติ จึงจำเป็นต้องกำหนดระดับความลึกของการค้นหาและใช้ฟังก์ชันการประเมินสถานะเพื่อประมาณความได้เปรียบของสถานะต่าง ๆ โดยฟังก์ชันการประเมินที่ดีควรสะท้อนโอกาสในการชนะของผู้เล่นได้อย่างเหมาะสม และสามารถคำนวณได้อย่างรวดเร็ว เพื่อให้ระบบปัญญาประดิษฐ์สามารถตัดสินใจได้ภายในเวลาที่กำหนด

นอกจากนี้ ตัวอย่างของเกมหิบบิหรือเกมโกะได้แสดงให้เห็นว่าหลักการของขั้นตอนวิธีมินิแมกซ์สามารถนำมาใช้วิเคราะห์และค้นหาการเดินที่เหมาะสมที่สุดได้อย่างเป็นระบบ โดยอาศัยการพิจารณาผลลัพธ์ที่อาจเกิดขึ้นในอนาคตและการวิเคราะห์ย้อนกลับจากสถานะปลายทางมายังสถานะเริ่มต้น แนวคิดดังกล่าวเป็นรากฐานสำคัญของการพัฒนาระบบปัญญาประดิษฐ์สำหรับเกมและการตัดสินใจในสถานการณ์การแข่งขัน

ปัจจุบัน การค้นหาแบบปฏิบัติไม่ได้ถูกนำไปใช้เฉพาะในเกมกระดานเท่านั้น แต่ยังถูกประยุกต์ใช้ในระบบปัญญาประดิษฐ์ด้านต่าง ๆ เช่น เกมคอมพิวเตอร์ การแข่งขันของหุ่นยนต์ การวางแผนเชิงกลยุทธ์ การบริหารทรัพยากร และการตัดสินใจในสถานการณ์ที่มีคู่แข่งหรือสภาพแวดล้อมที่เปลี่ยนแปลงอยู่ตลอดเวลา แม้ว่าปัจจุบันจะมีเทคนิคสมัยใหม่ เช่น การเรียนรู้เชิงลึก (Deep Learning) และการค้นหาแบบมอนติคาร์โล

รี (Monte Carlo Tree Search) เข้ามามีบทบาทมากขึ้น แต่แนวคิดพื้นฐานของการค้นหาแบบสุ่มยังคงมีอยู่
ขั้นตอนวิธีมินิแมกซ์ และการตัดกิ่งด้วยวิธีแอลฟา-บีตา ยังคงเป็นองค์ความรู้สำคัญที่เป็นรากฐานของ
ปัญญาประดิษฐ์สมัยใหม่ และเป็นพื้นฐานสำหรับการศึกษาเทคนิคขั้นสูงในบทถัดไป

แบบฝึกหัดท้ายบทที่ 3

1. จงอธิบายความหมายของการค้นหาแบบปฏิปักษ์ (Adversarial Search) และยกตัวอย่างปัญหาหรือเกมที่สามารถประยุกต์ใช้แนวคิดดังกล่าวได้
2. จงอธิบายองค์ประกอบของต้นไม้เกม (Game Tree) พร้อมทั้งอธิบายบทบาทของผู้เล่นฝ่าย MAX และผู้เล่นฝ่าย MIN ในขั้นตอนวิธีมินิแมกซ์ (Minimax Algorithm)
3. จงอธิบายหลักการทำงานของขั้นตอนวิธีมินิแมกซ์ (Minimax Algorithm) และอธิบายว่าทำไมขั้นตอนวิธีดังกล่าวจึงสามารถเลือกการเดินที่เหมาะสมที่สุดภายใต้สมมติฐานว่าคู่ต่อสู้มีเหตุผลและเลือกการเดินที่ดีที่สุดเสมอ
4. จงอธิบายความหมายของค่าแอลฟา (α) และค่าเบตา (β) พร้อมทั้งอธิบายหลักการของการตัดกิ่งด้วยวิธีแอลฟา-เบตา (Alpha-Beta Pruning) และประโยชน์ที่ได้รับจากการใช้เทคนิคดังกล่าว
5. จงอธิบายความจำเป็นของฟังก์ชันการประเมินสถานะ (Evaluation Function) ในเกมที่มีจำนวนสถานะเป็นจำนวนมาก เช่น หมากรุกหรือเกมโกะ และยกตัวอย่างปัจจัยที่อาจนำมาใช้ในการประเมินสถานะของเกม
6. จงอธิบายวิธีการแทนสถานะของปัญหาเกมหยิบเหรียญ (Coin Game) และอธิบายว่าผู้เล่นคนแรกควรเลือกหยิบเหรียญจำนวนเท่าใดจึงจะเพิ่มโอกาสในการชนะ พร้อมให้เหตุผลประกอบ
7. จงเปรียบเทียบข้อดีและข้อจำกัดของขั้นตอนวิธีมินิแมกซ์ (Minimax Algorithm) และขั้นตอนวิธีแอลฟา-เบตาพRUNING (Alpha-Beta Pruning)
8. จงอธิบายความสัมพันธ์ระหว่างต้นไม้เกม (Game Tree) ขั้นตอนวิธีมินิแมกซ์ (Minimax Algorithm) การตัดกิ่งด้วยวิธีแอลฟา-เบตา (Alpha-Beta Pruning) และฟังก์ชันการประเมินสถานะ (Evaluation Function) ในการสร้างระบบปัญญาประดิษฐ์สำหรับเกม
9. หากต้องการสร้างปัญญาประดิษฐ์สำหรับเกมหมากรุก จงเสนอปัจจัยที่ควรนำมาใช้ในการออกแบบฟังก์ชันการประเมินสถานะ (Heuristic Evaluation Function) พร้อมอธิบายเหตุผลของแต่ละปัจจัย
10. จงวิเคราะห์ข้อจำกัดของการค้นหาแบบปฏิปักษ์เมื่อประยุกต์ใช้กับเกมที่มีจำนวนสถานะมหาศาล และอธิบายว่าฟังก์ชันการประเมินสถานะและการตัดกิ่งด้วยวิธีแอลฟา-เบตาสามารถช่วยแก้ปัญหาดังกล่าวได้อย่างไร